

```
"""Survey Robot Rental & Sales - Python
backend (FastAPI + SQLite).
Run: pip install -r requirements.txt &&
uvicorn main:app --reload
Same API as before, so public/index.html
works unchanged. Swap sqlite3 for
PostgreSQL in production."""
import os, sqlite3, hashlib, hmac, secrets,
datetime as dt
import jwt
from fastapi import FastAPI, Depends,
HTTPException, Header, Body, Request
from fastapi.responses import JSONResponse
from fastapi.staticfiles import StaticFiles
from fastapi.exceptions import
RequestValidationError
from starlette.exceptions import
HTTPException as StarletteHTTPException

SECRET = os.environ.get("JWT_SECRET",
"change-me-in-production")
db =
sqlite3.connect(os.environ.get("DB_PATH",
"data.db"), check_same_thread=False)
db.row_factory = sqlite3.Row
db.execute("PRAGMA foreign_keys = ON")

db.executescript("""
CREATE TABLE IF NOT EXISTS users(id INTEGER
PRIMARY KEY, name TEXT, email TEXT UNIQUE,
```

```

org TEXT, phone TEXT,
    pass TEXT, role TEXT DEFAULT 'buyer',
created DATETIME DEFAULT
CURRENT_TIMESTAMP);
CREATE TABLE IF NOT EXISTS robots(id
INTEGER PRIMARY KEY, name TEXT, type TEXT,
description TEXT, environments TEXT,
    max_payload_kg REAL, battery_hours REAL,
buy_price REAL, day_rate REAL, active
INTEGER DEFAULT 1);
CREATE TABLE IF NOT EXISTS units(id INTEGER
PRIMARY KEY, robot_id INTEGER REFERENCES
robots(id), serial TEXT UNIQUE,
    status TEXT DEFAULT 'available');
CREATE TABLE IF NOT EXISTS sensors(id
INTEGER PRIMARY KEY, name TEXT, measures
TEXT, buy_price REAL, day_rate REAL,
weight_kg REAL);
CREATE TABLE IF NOT EXISTS
robot_sensors(robot_id INTEGER, sensor_id
INTEGER, PRIMARY KEY(robot_id, sensor_id));
CREATE TABLE IF NOT EXISTS requirements(id
INTEGER PRIMARY KEY, user_id INTEGER
REFERENCES users(id), industry TEXT,
    site_name TEXT, area_sqm REAL, terrain
TEXT, conditions TEXT, depth_m REAL,
measure_needs TEXT, duration_days INTEGER,
    mode TEXT, notes TEXT, status TEXT
DEFAULT 'new', created DATETIME DEFAULT
CURRENT_TIMESTAMP);
CREATE TABLE IF NOT EXISTS quotes(id
INTEGER PRIMARY KEY, requirement_id INTEGER
REFERENCES requirements(id),
    robot_id INTEGER REFERENCES robots(id),

```

```

sensor_ids TEXT, mode TEXT, days INTEGER,
amount REAL, deposit REAL,
    status TEXT DEFAULT 'sent', created_by
TEXT, created DATETIME DEFAULT
CURRENT_TIMESTAMP);
CREATE TABLE IF NOT EXISTS orders(id
INTEGER PRIMARY KEY, quote_id INTEGER
REFERENCES quotes(id), user_id INTEGER,
    unit_id INTEGER REFERENCES units(id),
start_date TEXT, end_date TEXT, status TEXT
DEFAULT 'pending_payment',
    paid REAL DEFAULT 0, tracking TEXT,
created DATETIME DEFAULT
CURRENT_TIMESTAMP);
CREATE TABLE IF NOT EXISTS reports(id
INTEGER PRIMARY KEY, order_id INTEGER
REFERENCES orders(id), title TEXT, url
TEXT,
    created DATETIME DEFAULT
CURRENT_TIMESTAMP);
CREATE TABLE IF NOT EXISTS maintenance(id
INTEGER PRIMARY KEY, unit_id INTEGER
REFERENCES units(id), note TEXT, cost REAL,
    created DATETIME DEFAULT
CURRENT_TIMESTAMP);
""")

# ----- helpers -----
def q(sql, *a): return [dict(r) for r in
db.execute(sql, a).fetchall()]
def one(sql, *a):
    r = db.execute(sql, a).fetchone();
return dict(r) if r else None
def run(sql, *a):

```

```

        cur = db.execute(sql, a); db.commit();
    return cur.lastrowid
def fail(msg, code=400): raise
HTTPException(code, msg)

def hash_pw(pw, salt=None):
    salt = salt or secrets.token_hex(8)
    return salt + "$" +
hashlib.pbkdf2_hmac("sha256", pw.encode(),
salt.encode(), 100_000).hex()
def check_pw(pw, stored):
    salt = stored.split("$")[0]; return
hmac.compare_digest(hash_pw(pw, salt),
stored)
def token(u): return jwt.encode({"id":
u["id"], "role": u["role"], "name":
u["name"],
    "exp": dt.datetime.utcnow() +
dt.timedelta(days=7)}, SECRET,
algorithm="HS256")

def auth(authorization: str = Header("")):
    try: return
jwt.decode(authorization[7:], SECRET,
algorithms=["HS256"])
    except Exception: fail("Login
required", 401)
def admin(user=Depends(auth)):
    if user["role"] != "admin": fail("Admin
only", 403)
    return user

# ----- seed demo data once -----
if not one("SELECT 1 FROM robots"):

```

```

def robot(*a): return run("INSERT INTO
robots(name,type,description,environments,max_payload_kg,battery_hours,buy_price,day_rate) VALUES(?,?,?,?,?,?,?,?,?)", *a)
def sensor(*a): return run("INSERT INTO
sensors(name,measures,buy_price,day_rate,weight_kg) VALUES(?,?,?,?,?)", *a)
r1 = robot("Crawler X1", "tracked
ground robot", "Rugged tracks for rubble,
mines and construction",
"dust,dark,water,gas", 25, 4, 9000, 120)
r2 = robot("Quad Scout", "drone",
"Indoor/outdoor quadcopter for mapping and
inspection", "dark,dust", 3, 0.5, 5500, 90)
r3 = robot("Hexa Trek", "legged robot",
"Legged robot for stairs, caves and uneven
terrain", "dark,dust,water", 15, 2.5,
18000, 220)
r4 = robot("Terra Rover", "wheeled
ground robot", "Long-range rover for
forests and open sites", "water,dust", 40,
8, 7000, 100)
s = [sensor("3D LiDAR",
"mapping,volume", 3500, 50, .9),
sensor("Thermal camera",
"thermal,hotspots", 2200, 35, .4),
sensor("Gas sensor array
(CH4/CO/H2S/O2)", "gas", 900, 25, .3),
sensor("RTK GPS", "positioning", 800, 15,
.2),
sensor("Ground-penetrating radar",
"subsurface,voids", 6500, 80, 4),
sensor("RGB 4K camera", "visual,cracks",
600, 10, .3),

```

```

        sensor("Multispectral camera",
"vegetation,forest health", 3000, 45, .5),
sensor("Dust/PM sensor", "air quality",
350, 8, .1)]
    for r, idx in {r1: [0, 1, 2, 5, 7], r2:
[0, 1, 3, 5, 6], r3: [0, 1, 2, 5, 7], r4:
[0, 1, 3, 4, 5, 6, 7]}.items():
        for i in idx: run("INSERT INTO
robot_sensors VALUES(?,?)", r, s[i])
        for i, r in enumerate([r1, r1, r2, r2,
r3, r4, r4]): run("INSERT INTO
units(robot_id,serial) VALUES(?,?)", r,
f"SN-{1000+i}")
        run("INSERT INTO
users(name,email,pass,role)
VALUES('Admin','admin@example.com',?,'admin
')", hash_pw("admin123"))

# ----- pricing & availability -----
---
def price(robot_id, sensor_ids, mode,
days):
    rb = one("SELECT * FROM robots WHERE
id=?", robot_id) or fail("Unknown robot")
    ok = {r["sensor_id"] for r in q("SELECT
sensor_id FROM robot_sensors WHERE
robot_id=?", robot_id)}
    sensors = []
    for sid in sensor_ids:
        if int(sid) not in ok:
fail(f"Sensor {sid} is not compatible with
{rb['name']}")
        sensors.append(one("SELECT * FROM
sensors WHERE id=?", sid))

```

```

        payload = sum(x["weight_kg"] for x in
sensors)
        if payload > rb["max_payload_kg"]:
fail(f"Payload {payload:.1f}kg exceeds
{rb['name']} limit of
{rb['max_payload_kg']}kg")
        buy_total = rb["buy_price"] +
sum(x["buy_price"] for x in sensors)
        if mode == "buy": return {"amount":
buy_total, "deposit": buy_total * 0.5,
"payload": payload}
        daily = rb["day_rate"] +
sum(x["day_rate"] for x in sensors)
        disc = 0.7 if days >= 28 else 0.85 if
days >= 7 else 1
        return {"amount": round(daily * days *
disc), "deposit": round(buy_total * 0.2),
"payload": payload}

def free_unit(robot_id, start, end):
    return one("""SELECT u.* FROM units u
WHERE u.robot_id=? AND
u.status!='maintenance' AND u.id NOT IN (
        SELECT unit_id FROM orders WHERE
unit_id IS NOT NULL AND status NOT
IN('cancelled','completed','returned')
        AND start_date<=? AND end_date>=?)
LIMIT 1""", robot_id, end, start)

def suggest(b):
    conds, needs = b.get("conditions") or
[], b.get("measure_needs") or []
    out = []
    for rb in q("SELECT * FROM robots WHERE

```

```

active=1"):
    env_ok = sum(c in
rb["environments"].split(",") for c in
conds)
    sens = [x for x in q("SELECT s.*
FROM sensors s JOIN robot_sensors rs ON
rs.sensor_id=s.id WHERE rs.robot_id=?",
rb["id"])]
        if any(n in x["measures"]
for n in needs)]
            if sens: out.append({"robot": rb,
"score": env_ok * 2 + len(sens),
"sensor_ids": [x["id"] for x in sens]})
                for o in out: o["robot"]["sensor_ids"]
= [r["sensor_id"] for r in q("SELECT
sensor_id FROM robot_sensors WHERE
robot_id=?", o["robot"]["id"])]
                    return sorted(out, key=lambda x: -
x["score"])[ :3]

# ----- app -----
app = FastAPI(title="Survey Robot Rental")

@app.exception_handler(StarletteHTTPExcepti
on)
async def _http(_: Request, e): return
JSONResponse({"error": e.detail},
status_code=e.status_code)
@app.exception_handler(RequestValidationErr
or)
async def _val(_: Request, e): return
JSONResponse({"error": "Invalid input"},
status_code=422)

```

```

# auth
@app.post("/api/auth/register")
def register(b: dict = Body(...)):
    if not b.get("email") or
len(b.get("password") or "") < 6:
fail("Email and a 6+ character password are
required")
    if one("SELECT 1 FROM users WHERE
email=?", b["email"]): fail("That email is
already registered", 409)
    uid = run("INSERT INTO
users(name,email,org,phone,pass)
VALUES(?,?,?,?,?)", b.get("name"),
b["email"], b.get("org"), b.get("phone"),
hash_pw(b["password"]))
    u = {"id": uid, "role": "buyer",
"name": b.get("name")}
    return {"token": token(u), "name":
u["name"], "role": "buyer"}

@app.post("/api/auth/login")
def login(b: dict = Body(...)):
    u = one("SELECT * FROM users WHERE
email=?", b.get("email"))
    if not u or not
check_pw(b.get("password") or "",
u["pass"]): fail("Wrong email or password",
401)
    return {"token": token(u), "name":
u["name"], "role": u["role"]}

# catalog
@app.get("/api/robots")
def robots():

```

```

        rows = q("SELECT * FROM robots WHERE
active=1")
        for r in rows:
            r["sensor_ids"] = [x["sensor_id"]
for x in q("SELECT sensor_id FROM
robot_sensors WHERE robot_id=?", r["id"])]
            r["units_total"] = one("SELECT
COUNT(*) c FROM units WHERE robot_id=?",
r["id"])[ "c"]
        return rows

@app.get("/api/sensors")
def sensors(): return q("SELECT * FROM
sensors")

@app.get("/api/price")
def get_price(robot: int, mode: str =
"rent", days: int = 1, sensors: str = ""):
    return price(robot, [int(x) for x in
sensors.split(",") if x], mode, days)

@app.get("/api/availability")
def availability(robot: int, start: str,
end: str): return {"available":
bool(free_unit(robot, start, end))}

# requirements & quotes
@app.post("/api/requirements")
def add_req(b: dict = Body(...),
user=Depends(auth)):
    if not b.get("industry") or not
b.get("measure_needs"): fail("Choose an
industry and what you need measured")
    rid = run("""INSERT INTO

```

```

requirements(user_id,industry,site_name,area_sqm,terrain,conditions,depth_m,measure_needs,duration_days,mode,notes)
        VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?)""",
user["id"], b["industry"],
b.get("site_name"), b.get("area_sqm") or
None, b.get("terrain"),
        ",".join(b.get("conditions") or
[]), b.get("depth_m") or None,
        ",".join(b["measure_needs"]),
        int(b.get("duration_days") or 1),
b.get("mode", "rent"), b.get("notes"))
    return {"id": rid, "suggestions":
suggest(b)}

@app.get("/api/requirements")
def list_req(user=Depends(auth)):
    if user["role"] == "admin":
        return q("SELECT r.*,u.name
buyer,u.org FROM requirements r JOIN users
u ON u.id=r.user_id ORDER BY r.id DESC")
    return q("SELECT * FROM requirements
WHERE user_id=? ORDER BY id DESC",
user["id"])

@app.post("/api/requirements/{rid}/quote")
def make_quote(rid: int, b: dict =
Body(...), user=Depends(auth)):
    req = one("SELECT * FROM requirements
WHERE id=?", rid)
    if not req or (user["role"] != "admin"
and req["user_id"] != user["id"]):
fail("Not found", 404)
    mode, days, sids = b.get("mode",

```

```

req["mode"]), b.get("days",
req["duration_days"]), b.get("sensor_ids")
or []
    p = price(b.get("robot_id"), sids,
mode, days)
    amount = b["amount"] if user["role"] ==
"admin" and b.get("amount") else
p["amount"] # admin can override
    qid = run("INSERT INTO
quotes(requirement_id,robot_id,sensor_ids,m
ode,days,amount,deposit,created_by)
VALUES(?,?,?,?,?,?,?,?)",
            rid, b["robot_id"],
", ".join(map(str, sids)), mode, days,
amount, p["deposit"], user["role"])
    run("UPDATE requirements SET
status='quoted' WHERE id=?", rid)
    return {"id": qid, "amount": amount,
"deposit": p["deposit"]}

@app.get("/api/quotes")
def list_quotes(user=Depends(auth)):
    where, args = ("1=1", []) if
user["role"] == "admin" else
("rq.user_id=?", [user["id"]])
    return q(f""""SELECT q.*,rb.name robot
FROM quotes q JOIN requirements rq ON
rq.id=q.requirement_id
JOIN robots rb ON rb.id=q.robot_id
WHERE {where} ORDER BY q.id DESC""", *args)

# orders
@app.post("/api/quotes/{qid}/accept")
def accept(qid: int, b: dict = Body(...),

```

```

user=Depends(auth)):
    qt = one("SELECT q.*,rq.user_id FROM
quotes q JOIN requirements rq ON
rq.id=q.requirement_id WHERE q.id=?", qid)
    if not qt or qt["user_id"] !=
user["id"]: fail("Not found", 404)
    if qt["status"] != "sent": fail("Quote
already handled")
    start = end = None
    if qt["mode"] == "rent":
        start = b.get("start_date") or
fail("Pick a start date")
        end = (dt.date.fromisoformat(start)
+ dt.timedelta(days=qt["days"] -
1)).isoformat()
        unit = free_unit(qt["robot_id"],
start, end)
    else:
        unit = one("SELECT * FROM units
WHERE robot_id=? AND status='available'
LIMIT 1", qt["robot_id"])
        if not unit: fail("No unit available
for those dates. Try another date or ask us
for a custom quote.", 409)
        oid = run("INSERT INTO
orders(quote_id,user_id,unit_id,start_date,
end_date) VALUES(?,?,?,?,?)", qid,
user["id"], unit["id"], start, end)
        run("UPDATE quotes SET
status='accepted' WHERE id=?", qid)
        return {"order_id": oid, "due_now":
qt["deposit"]}

@app.post("/api/orders/{oid}/pay")

```

```

def pay(oid: int, user=Depends(auth)): #
STUB: integrate Razorpay/Stripe here and
confirm via webhook
    o = one("SELECT o.*,qt.deposit FROM
orders o JOIN quotes qt ON qt.id=o.quote_id
WHERE o.id=? AND o.user_id=?", oid,
user["id"])
    if not o: fail("Not found", 404)
    run("UPDATE orders SET
paid=?,status='confirmed' WHERE id=?",
o["deposit"], oid)
    return {"ok": True}

@app.get("/api/orders")
def list_orders(user=Depends(auth)):
    where, args = ("1=1", []) if
user["role"] == "admin" else
("o.user_id=?", [user["id"]])
    return q(f""SELECT
o.*,qt.amount,qt.mode,qt.sensor_ids,rb.name
robot,un.serial,u.name buyer FROM orders o
JOIN quotes qt ON qt.id=o.quote_id
JOIN robots rb ON rb.id=qt.robot_id LEFT
JOIN units un ON un.id=o.unit_id
JOIN users u ON u.id=o.user_id
WHERE {where} ORDER BY o.id DESC"", *args)

@app.get("/api/orders/{oid}/reports")
def order_reports(oid: int,
user=Depends(auth)):
    o = one("SELECT user_id FROM orders
WHERE id=?", oid)
    if not o or (user["role"] != "admin"
and o["user_id"] != user["id"]): fail("Not

```

```

found", 404)
    return q("SELECT * FROM reports WHERE
order_id=?", oid)

# admin
@app.patch("/api/admin/orders/{oid}")
def set_order(oid: int, b: dict =
Body(...), _=Depends(admin)):
    o = one("SELECT * FROM orders WHERE
id=?", oid) or fail("Not found", 404)
    run("UPDATE orders SET
status=COALESCE(?,status),tracking=COALESCE
(?,tracking) WHERE id=?", b.get("status"),
b.get("tracking"), oid)
    if b.get("status") == "delivered":
run("UPDATE units SET status='rented' WHERE
id=?", o["unit_id"])
    if b.get("status") in ("returned",
"completed"): run("UPDATE units SET
status='available' WHERE id=?",
o["unit_id"])
    return {"ok": True}

@app.post("/api/admin/reports")
def add_report(b: dict = Body(...),
_ = Depends(admin)):
    return {"id": run("INSERT INTO
reports(order_id,title,url) VALUES(?,?,?)",
b["order_id"], b["title"], b["url"])}

@app.post("/api/admin/robots")
def add_robot(b: dict = Body(...),
_ = Depends(admin)):
    rid = run("INSERT INTO

```

```

robots(name,type,description,environments,max_payload_kg,battery_hours,buy_price,day_rate) VALUES(?,?,?,?,?,?,?,?)",
        b["name"], b.get("type"),
b.get("description"),
b.get("environments"),
b.get("max_payload_kg"),
b.get("battery_hours"), b["buy_price"],
b["day_rate"])
        for s in b.get("sensor_ids", []):
run("INSERT OR IGNORE INTO robot_sensors
VALUES(?,?)", rid, s)
        return {"id": rid}

@app.post("/api/admin/sensors")
def add_sensor(b: dict = Body(...),
_=_Depends(admin)):
        return {"id": run("INSERT INTO
sensors(name,measures,buy_price,day_rate,weight_kg) VALUES(?,?,?,?,?)",
        b["name"],
b.get("measures"), b["buy_price"],
b["day_rate"], b.get("weight_kg", 0))}

@app.post("/api/admin/units")
def add_unit(b: dict = Body(...),
_=_Depends(admin)):
        return {"id": run("INSERT INTO
units(robot_id,serial) VALUES(?,?)",
b["robot_id"], b["serial"])}

@app.post("/api/admin/maintenance")
def add_maintenance(b: dict = Body(...),
_=_Depends(admin)):

```

```

        run("INSERT INTO
maintenance(unit_id,note,cost)
VALUES(?,?,?)", b["unit_id"],
b.get("note"), b.get("cost"))
        run("UPDATE units SET status=? WHERE
id=?", "available" if
b.get("back_in_service") else
"maintenance", b["unit_id"])
        return {"ok": True}

@app.get("/api/admin/stats")
def stats(_=Depends(admin)):
    return {"new_requirements": one("SELECT
COUNT(*) c FROM requirements WHERE
status='new'")["c"],
            "active_orders": one("SELECT
COUNT(*) c FROM orders WHERE status NOT
IN('completed','cancelled')")["c"],
            "revenue": one("SELECT
COALESCE(SUM(paid),0) s FROM orders")["s"],
            "units": q("SELECT
status,COUNT(*) c FROM units GROUP BY
status"))}

app.mount("/",
StaticFiles(directory="public", html=True),
name="static") # must stay last

```